Date 28/2/26

UNIT - IV

State Graphs and Transition Testing

State Graph:- The behaviour of the system is represented in a graphical form which is known as state graph. The state graph is used for functional testing of the system to identify different bugs

The different bugs are:

state bugs & Transition bugs

The state graph consisting of mainly 4 elements

→ states
→ Input
→ Transition
→ output

The states:- A state in a state graph is the node which represents the characteristics and behaviour of the system, it has set of states, they are identified by characters or no's

I/P:- The state in the state graph contains i/p value to represent the changes to the states

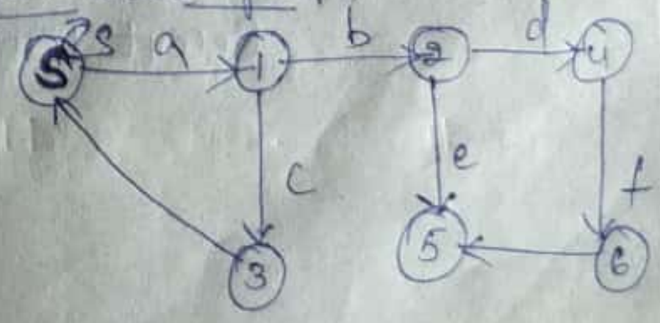
→ They can be denoted by no. of characters.

The transitions:- The link that joins two states is known as transition

→ When some i/p is applied to the node the state changes and the processing of changing of state is known as Transition.

Output:- After processing input the result is produced which is known as o/p. The input and the output of a state graph are represented by slash.

Example for state graph



The states & inputs and transition in the state graph can be explain as

S → start state

a → Enter pin number

1 → Verification of Pin number

c → Pin number is wrong

3 → re enter Pin

b → Pin number is OK

2 → screen display & options
Balance enquery
Money withdraw

e → if balance energy is chosen

5 → Print receipt

d → Money withdrawal is chosen

4 → enter amount

f → amount is entered

6 → Received money
Receipt Printed

2/3/26 ~~Finite state Ha~~

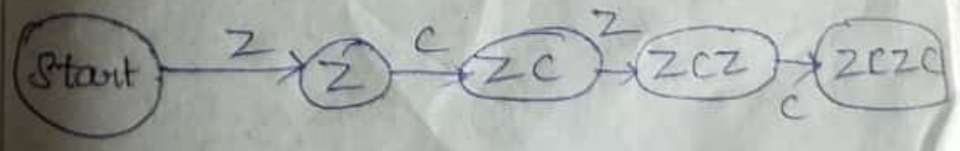
The state graph consisting of number of states and number of inputs whenever we are providing input for the particular state it moves to another state by using the transition finally it generates the o/p

Eg.1 :- A moving automobile whose engine is running can have the following states w.r.t it's transitions

The following states are
Neutral gear
Reverse gear
1st, 2nd, 3rd, 4th, 5th gears

Ex:-2 The program detects the character sequence ZCZC can be following states

→ The ZCZC can be detected
i) Z has been detected
ii) ZC is detected
iii) ZCZ is detected
iv) ZCZC is detected



Finite state Machine :- It is a model of computation consisting of a set of states, a start state an input alphabet and transition function that maps i/p symbol and current state to next state.

The finite state machine can model a large number of models

i) Design automation

ii) Parsing

iii) Protocol design

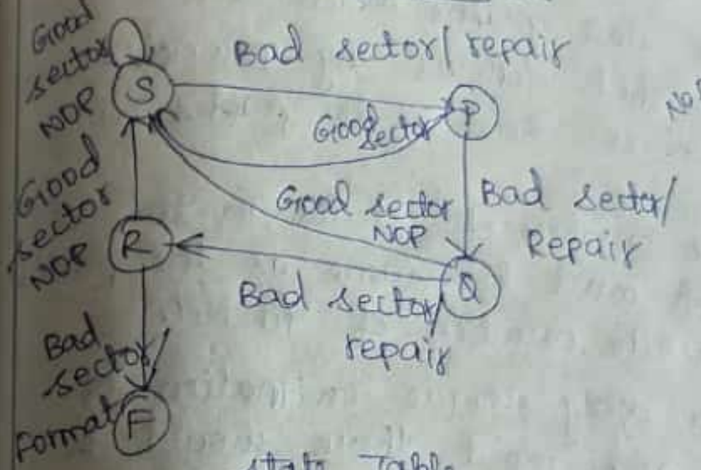
state tables:- The representation of state graph in the form of tables is known as state tables.

→ If the graph is converted into tables it could be more convenient.

→ The state table is also known as state transition table.

→ The table contains rows and columns to store information.

Construct state graph for hard disk recovery process



state Table

state / Inputs	Good sector	Bad sector
S	S/NOP	P/repair
P	S/NOP	Q/repair
Q	S/NOP	R/repair
R	S/NOP	F/format
F	–	–

Date 6/3/26

Good and Bad state Graphs

Good state graphs:-

The state graphs which can be good and which follows any principles are known as "Good state graphs". They are

→ The number of states in the state graph must be same as the product of possible number of factors

→ For every unique combination of state and input, there should be one transition specified.

→ For every transition, there should be atleast one sensible output.

State graphs

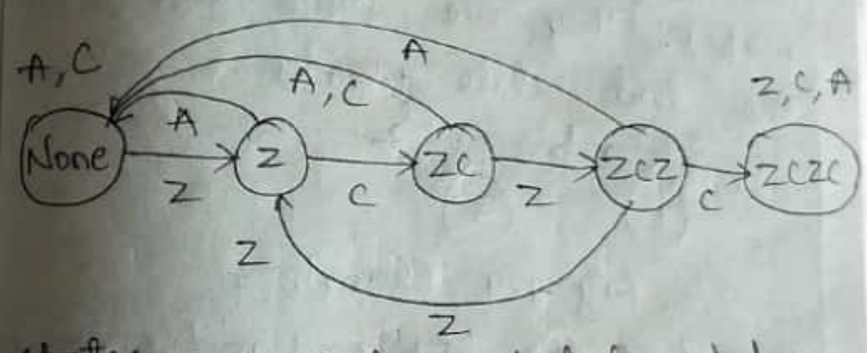
· A state is defined as "A combination of circumstances or attributes belonging for the time being to a person or thing."

→ For example, a moving automobile whose engine is running can have the following states with respect to its transmission.

· Reverse gear
· Neutral gear
· First gear
· Second gear
· Third gear
· Fourth gear state graph- Example

· For example, a program that detects the character sequence "ZCZC" can be in the following states.

· Neither ZCZC nor any part of it has been detected.
· Z has been detected
· ZC has been detected
· ZCZ has been detected
· ZCZC has been detected.



states are represented by Nodes.
state are numbered or may identified by words or whatever else is convenient.

Inputs and Transitions

· Whatever is being modeled is subjected to inputs. As a result of those inputs, the state changes, or is said to have made a transition.

→ Transitions are denoted by links and

that join the states.
→ The input that causes the transition are marked on the link; that is, the inputs are link weights.
→ There is one out link from every state for every input.

Bad state graphs

The state graphs which cannot be good and which doesn't follows any principles are known as "Bad state graphs". They are some cases to show the bad state graphs.

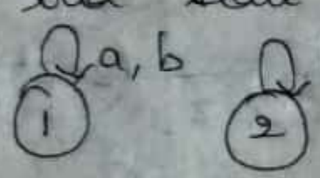


Fig 7.9 Example 3

Here both the states 1 and 2 are not reachable as there is no path between them.

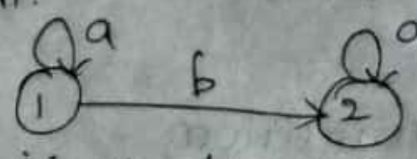


Figure 7.10 Example 4

There is no transition for input b at state '2'. If input 'b' is received at state '2' the path is undefined.

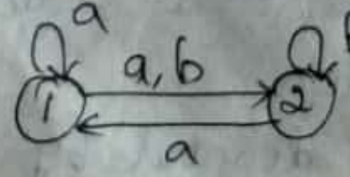


Figure 7.11 Example 5

There are two different paths for the same input on same state. The state will be in confusion to consider the path when input 'a' is given at state 1.

Comparison between good and bad state graphs

Table 7.2 Comparison between good and bad state graphs.

	Good state graph	Bad state graph
①	If the elements of the state graph i.e., inputs, transitions, outputs are clearly specified and they are easily understandable, then they are said to be good state graphs.	→ If the elements of the state graph are not specified clearly then they are said to be bad state graphs
②	In a good state graph, every state should have a sequence of inputs that derives the system to the starting state	② In a bad state graph, the sequence of inputs does not lead to the initial state
③	There should be only one transition specified	More than one transitions are

	for one state and input combination	specified for every state and input combination
4	There would be only one output for every transition because unique output gives clarity otherwise it leads to confusion	→ There might be none or more than one output for every transition which results in improper state graph
5	Here bugs are less and they are easy to find.	Here bugs are more and difficult to find

STATE BUGS

The reasons for which state bugs arise are,

1. Number of states
2. Impossible states
3. Equivalent states

Number of states :-

→ The number of states in the state graph can be considered as the number of states they are required to identify the behaviour of the system.

→ Every state of the graph is very important to the state testing.
→ So, it is essential to find the number of states

The number of states can be identified by,
→ Finding all factors of the state
→ Finding all the possible values of each factor.

Impossible states

→ There can be some combinations of factors that may appear to be impossible. If these factors are included, they lead to impossible states.
→ The impossible states do not have any meaning in the systems as the impossible states are the one which are not possible to occur.

Table 7.3 Example for Impossible states

Direction	Forward, Reverse, Stop	= 3 factors
Gear	R, N, 1, 2, 3, 4, 5	= 7 factors
Engine operation	Running, stopped	= 2 factors
Engine Condition	good, faulty	= 2 factors
Total		84 states

Equivalent states

→ If every sequence of inputs from one state generates exactly the same sequence of outputs of other states, then both the states are said to be equivalent states.

→ When both the states produce same output then only one state would be enough instead of two states.

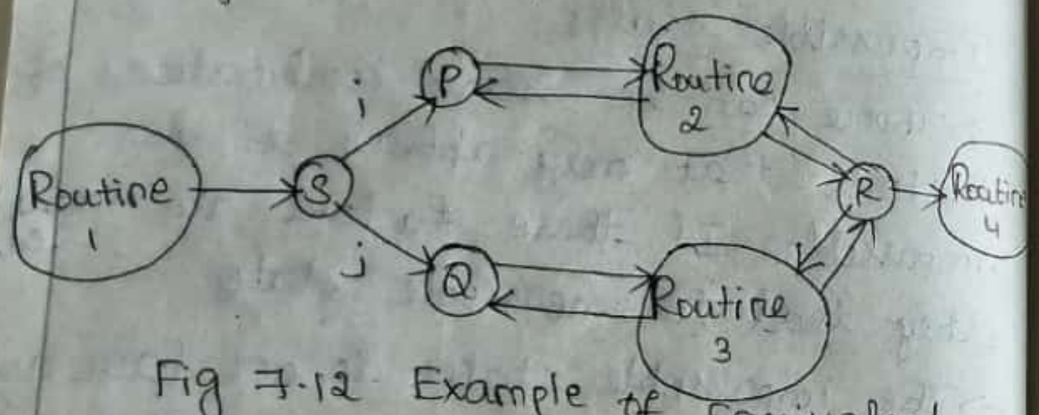


Fig 7.12 Example of Equivalent states

Transition Bugs

The transition can be caused due to

- Unspecified and contradictory transitions
- Unreachable states
- Dead states

Unspecified and Contradictory Transitions

For every state input combination, there should be a transition specified. If the transitions are not specified then the bugs can occur due to unspecified transitions between the states.

Unreachable states

→ If there is some state which no input sequence can reach then the state is known as "unreachable state"

→ There are transitions from unreachable states as incorrect transitions.

Dead states :-

Dead state is such a state that if it is entered once, that state cannot be left. The inputs cannot be processed from the dead states.

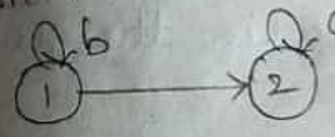


Fig. 7.18 Dead state graph.

Output errors :-

→ The output of the transitions could be wrong even if all the states, the transitions and inputs are correct and there are no dead or unreachable states.

→ The reason for the wrong output could be incorrect call to the routine that

should be executed for output. This is a minor bug.

Encoding bugs:-

The process of converting or coding the inputs, transitions and outputs of the states in order to obtain the secure process of finite state machine is known as Encoding and it is essential process.

→ Emplict is done both in explicit and implicit way.

State Testing

→ state testing is defined as a functional testing technique to test the functional bugs in the entire system.

→ The principles for state testing are very similar to the principles for path testing.

→ For path-testing it is not possible to test every possible path in a flow graph.

→ Similarly for state testing, it is not possible to test every possible path in a state graph.

Advantages

→ state testing is useful when the error corrections are less expensive.

→ state testing is also useful when the testers want to detect a specified input.

→ A state testing provides the bigged rewards during the design of the tests.

Disadvantages

→ state testing does not provide thorough testing because when a test is completed, there might be some bugs remained into the system

→ Testers require large number of input sequences to catch transition errors, missing states, extra states and the link.

Implementation issues

→ The bugs in the software can occur through many different cases.

→ Due to these bugs it is impossible to cover all the paths in the state graph.

The bugs may occur due to
* Incorrect number of states
* Incorrect state transitions
* Incorrect transition output
* Dead states
* Unreachable states.

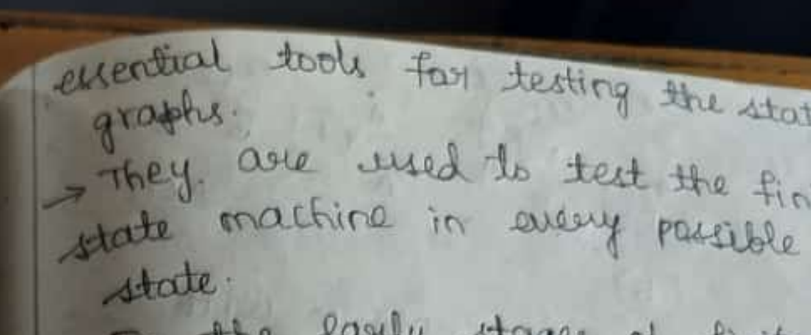
→ The occurence of bugs will be more in strongly connected graph.
→ When the system has no state and when there are improper responses the state diagrams are not applied.
→ When each and every thing is clearly specified such as transitions and outputs for each state input combination, it would be easy to catch bugs.
→ state testing is considered as bottom line method as it supports functional model rather than the structural details.

Testability tips

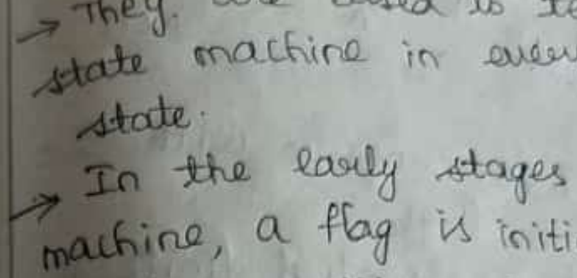
Switches, flags and unachievable paths

The switches and flags are most essential tools for testing the state graphs.
→ They are used to test the finite state machine in every possible state.
→ In the early stages of finite state machine, a flag is initialized and depending on this value a specific path is selected to find the easiest path for testing the finite state machine.

Consider the example of a program with one flag variable as shown in figure.

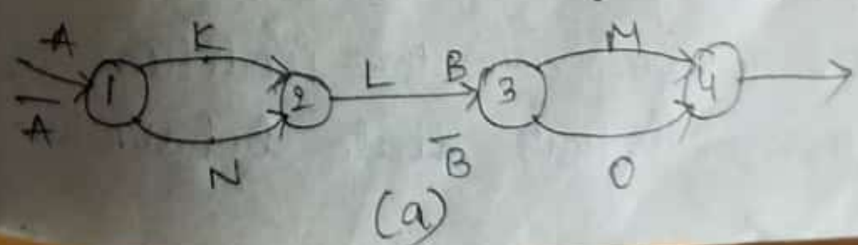


(a)

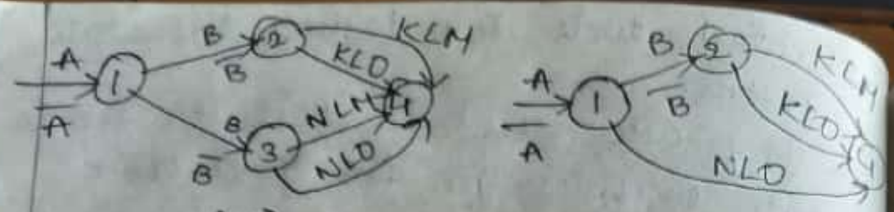


(b)

Fig. 7.19 Program with one flag



(a)



(b)

Fig 7.20 Program with two flags.

Essential and Inessential finite state Behaviour.

→ Essential finite state machine is purely based on acknowledgement and has no specific logic for implementation without acknowledgement.

→ Even decision table or tree is unable to classify its behaviour until acknowledgement is stored in a table.

→ The only problem with the essential finite machine behaviour is that it provides lots of work i.e., just for the sake of branch testing, every state must be tested which is not mandatory or essential.

The following are the conditions of inessential finite state behaviour.

If these conditions are not satisfied then the behaviour of finite state machine is known as essential.

1) If behaviour can be obtained by parallel program in data flow machine
2) If behaviour is obtained from decision table or decision tree.
3) If expression of exit is equal to unity of a program including loops.
4) If the program's exit expression is not equal to unity but we don't want to loop under the looping conditions.

Design Guidlines for building finite machine into your code

→ How implementation of FSM done both in software and hardware.
→ start the designing and check it for the requirements.
→ Carry out the analysis in the form of a state graph or state table with three or more states
→ Design the state machine explicitly i.e., clear and understandable.
→ This test must be conducted throughly

to determine the processing time and space of explicit.

→ If time or space is efficient the overall system, then use shortcuts to complete the design process.

→ If there are more than dozen states than construct a hierarchial design.

Date
13/2/26

→ The g
another
graph mat
→ The ma
code are
→ Matrix
animation
→ Matrix
architectu
→ Math
Matrix
Matrix
The are
graphs
Each and
represen
column
→ Some
matrix
a)